Tutorial Sheet (Module-2)

Course: B.Tech (CSE) Year: III Semester: V Session: 2025-26

Subject Code & Name: BCSC-0011 (Theory of Automata & Formal Language)

Topic – LMD, RMD, Derivation Tree/Parse Tree

1. Construct both the left-most and right-most derivation trees for the string (abbba)
   S→aAB|bBA
   A→aA|b B→bB|a
2. S→AB|BA A→aA|a B→bB|b
   Provide the left-most and right-most derivation sequences for the string (aabbb)
3. Let G be the grammar S→aB|bA A→a|aS|bAA B→b|bS|aBB. For the string aaabbabbba, find (a) LMD (b) RMD (c) Derivation Tree.
4. Let the grammar is given as
   S → AB|aaB
   A → a| Aa
   B → bS|b
   For the string aabaab, find (a) LMD (b) RMD (c) Derivation Tree.

5. Consider the following productions:
   $S \rightarrow aAB \mid a$
   $A \rightarrow aBA \mid bAA \mid a$
   $B \rightarrow aBB \mid b$
   For the string aabab generate:

i) Leftmost Derivation
ii) Rightmost Derivation
iii) Parse Tree or Derivation Tree

Topic – Ambiguity

6. S→aSb|SS|ϵ
   Is this grammar ambiguous? If yes, show how it generates ambiguity using the string aabb.

7. E→E+E|E∗E|(E)|id
   Prove that this grammar is ambiguous by showing different parse trees for the expression id + id * id.

8. Suppose a tiny robot language encodes simple commands where the letter a means "step forward" and b means "beep". A valid program for the robot must end with a beep (b). The grammar G that generates robot programs is:

$S \rightarrow AB|aaB$

$A \rightarrow a|\ Aa$

$B \rightarrow b$

Determine whether grammar G is ambiguous or not.

9. A group of computer science researchers at SyntaxAI Labs is working on building a syntax analyzer for a programming language parser. During testing, one of the interns designed a Context-Free Grammar (CFG) to represent a simple pattern of strings containing a's followed by b's. However, the lead engineer suspects that the grammar may be ambiguous, meaning that some strings generated by this grammar might have more than one parse tree or different leftmost derivation. The grammar designed by the intern is given as:

S -> SS|AB , A -> Aa|a , B -> Bb|b

Determine whether grammar G is ambiguous or not.

Topic – Simplification of Grammar

10. Convert the following grammar into an equivalent one with no unit productions and no useless symbols

$S \rightarrow ABA$

$A \rightarrow aAA|aBC|bB$

$B \rightarrow a|bB|Cb$

$C \rightarrow CC|Cc$

11. Convert the following grammar into an equivalent one with no unit productions and no useless symbols

$S \rightarrow ABC \mid BaB$

$A \rightarrow aA|BaC|aaa$

$B \rightarrow bBb|a$

$C \rightarrow CA|AC$

12. $S \rightarrow AB|Ba$

$A \rightarrow a$

$B \rightarrow C$

$C \rightarrow bC|\epsilon$

Simplify the above context free grammar.

13. The software company "ParseTech Solutions" is developing a syntax optimization engine for simplifying grammars used in language compilers. During one of the grammar analysis sessions, the team encounters a complex Context-Free Grammar (CFG) that generates combinations of symbols a, b,

and c. However, the grammar contains unnecessary productions and intermediate variables, making it difficult to analyze and process. The original grammar, with S as the start symbol, is given below:

S→AB|C
A→aA|B
B→b
C→cC|D
D→ϵ

As a member of the research and development team, you are tasked with simplifying this grammar to make it more efficient while ensuring it generates the same language.

14. A software company is developing a syntax analyzer for a compiler project named "LangLite". During the grammar optimization phase, one of the interns designed a context-free grammar (CFG) for a prototype language. However, the grammar turned out to be inefficient because it contains useless symbols — i.e., symbols that either cannot derive any terminal string or cannot be reached from the start symbol. The grammar (with S as the start symbol) is given as:

$S \rightarrow AB|AC$
$A \rightarrow aAb \mid bAa \mid a$
$B \rightarrow bbA \mid aaB \mid AB$
$C \rightarrow abCa \mid aDb$
$D \rightarrow bD|aC$

Eliminate all useless symbols from the grammar and write an equivalent CFG.

Topic – Chomsky Normal Form

15. Convert the following grammar into Chomsky normal form.

$S \rightarrow a|abSb|aAb, \qquad A \rightarrow bS|aAAb$

16. S→ABa|a
    A→BCB|b
    B→b
    C→c
    Convert the above grammar to CNF
17. S→aA
    A→Bb
    B→b
    Convert the above grammar to CNF
18. Construct the CNF for the following grammar and explain the steps.
    S→aAa | bBb
    A→C|a
    B→C|b
    C→CDE | €

D→A|B|ab

19. Convert the following grammar into CNF
    S→cBA, S→A, A→cB, A→AbbS, B→aaa
20. Convert the following grammar into CNF
    S→a|AAB, A→ab|aB| ε, B→aba| ε
21. Convert the following grammar into CNF
    S→A|CB, A→C|D, B→1B|1, C→0C|0, D→2D|2
22. The research team at AutomataTech Labs is developing a grammar transformation module for a compiler design project. The goal of the module is to convert any given Context-Free Grammar (CFG) into its equivalent Chomsky Normal Form (CNF) so that it can be used for efficient parsing using algorithms like the CYK (Cocke–Younger–Kasami) parser. During testing, the team provides the following grammar as an input to the module, with S as the start symbol:
    S→aAD A→aB|bAB B→ b D→ d
    Your task as a grammar engineer in the team is to perform manual conversion of the given grammar into its Chomsky Normal Form (CNF) so that it can be verified against the tool's output.

Topic – Greibach Normal Form

23. Find Greibach normal form for the following grammar S→AA | 1, A→SS |0
24. Find Greibach normal form for the following grammar S→a|AB, A→a|BC, B→b, C→b
25. Find Greibach normal form for the following grammar A1→A2A3, A2→A3A1|b, A3→A1A2|a
26. S→S+S|a
    Convert the above grammar to GNF
27. S→A|AB
    A→aA|ϵ
    B→bB|b
    Convert the above grammar to GNF
28. Convert the following grammar G in Greibach normal form
    S→ABb|a A→aaA|B B→bAb
29. Convert the grammar $A \rightarrow BB \mid 0,\ B \rightarrow AA \mid 1$ into GNF.

Topic – Grammar to PDA Conversion

30. Find the PDA equivalent to the given CFG
    S → 0S1 | A
    A → 1A0 | S | ε
31. Find the PDA equivalent to the given CFG

S → 0BB
B → 0S | 1S | 0

32. Find the PDA equivalent to the given CFG with the following productions
S→A, A→BC, B→ba, C→ac
33. Find the PDA equivalent to the given CFG with the following productions
S→aSb|A, A→bSa|S| ε
34. Find the PDA equivalent to the given CFG
S → XY
X → aX | ε
Y → bY c | ε

Topic – PDA to Grammar Conversion

35. Consider the given PDA M=({q0}, {0,1}, {a,b,Z0}, δ, q0, Z0, Ø)
Where δ is defined as follows:
δ (q0 , 0, Z0)= {(q0, aZ0)}
δ(q0,1,Z0)={(q0, bZ0)}
δ (q0 , 0, a)= {(q0, aa)}
δ (q0 ,1, b)= {(q0, bb)}
δ (q0 , 0, b)= {(q0, ε)}
δ (q0 , 1, a)= {(q0, ε)}
δ (q0 , ε, Z0)= {(q0, ε)}
Convert the PDA M into equivalent Grammar.
36. Convert the PDA M = ({q0,q1}, {0,1}, {X, Zo}, ϵ, q0, Zo, {}) into an equivalent CFG, where δ of PDA is given as:
δ(q0,0,Z0) = (q0,XZo)
δ(q0,0,X) = (q0,XX)
δ(q0,1,X) = (q1,ϵ)
δ(q1,1,X) = (q1,ϵ)
δ(q1,ϵ,X) = (q1,ϵ)
δ(q1,ϵ,Zo) = (q1,ϵ)
37. Find a context-free grammar that generates the language accepted by the npda M = ({q0, q1}, {a, b}, {A, z}, δ, q0, z, {q1}), with transitions
δ (q0, a, z) = {(q0,Az)},
δ (q0, b,A) = {(q0,AA)},
δ (q0, a,A) = {(q1, λ)}.
38. Construct the equivalent CFG (G) for the given PDA (A).
A = ({q0, q1}, {0, 1}, {X, Zo}, d, q0, Z0, {})
δ (q0, 0, Z0) = (q0, XZ0)
δ (q0, 1, X) = (q1, ^)
δ (q1, 1, X) = (q1, ^)

39. An automata linguist is studying a rare ancient script inscribed on a wooden piece found in Mathura. The script consists of sequences of symbols 'a' and 'b', and follows a unique stacking pattern during inscription. The scribe uses a special stack starting with a base marker Z0. The inscription and process works are given as follows:
    A = ({q0, q1}, {a, b}, {a, Z0}, δ, q0, Z0, {})
    δ (q0, a, Z0) = (q0, aZ0)
    δ (q0, a, a) = (q0, aa)
    δ (q0, b, a) = (q1, a)
    δ (q1, b, a) = (q1, a)
    δ (q1, a, a) = (q1, ^)
    δ (q1, ^, Z0) = (q1, ^)
    But as an automata linguist, you are required to convert that script into equivalent CFG to decode it. Hence it's your job now to convert it into CFG and make it readable.

Topic – Turing Machine Construction

40. Construct a Turing machine to compute 'n mod 2' where n is represented in the tape in unary form consisting of only 0's.
41. Design a TM that accepts the language of odd integers written in binary.
42. Design a Turing machine with no more than three states that accepts the language a(a+b)*.
43. Design a Turing machine for the Reverse the given string {abb}
44. Design a Turing machine for the $L=\{a^nb^nc^n \mid n>0\}$
45. Design a Turing machine to perform proper subtraction.
46. Design a Turing machine for the langauge $L=\{1^n0^n \mid n>=1\}$
47. Design a Turing machine for the langauge $L=\{ww^R \mid w \text{ is in } (0+1)^*\}$
48. Design a Turing machine for the langauge $L=\{wcw^R \mid w \text{ is in } (0+1)^*\}$
49. Design a Turing machine for the langauge $L=\{wcw^R \mid w \text{ is in } (0+1)^*, c \in \{0,1\}\}$
50. A research lab named "Symbolic Computation Systems" is developing a pattern-verification module for a compiler prototype. The module must verify whether an input string follows a balanced pattern of symbols, where every occurrence of a is perfectly matched by a corresponding b, in the same order. For example, the strings "ab", "aabb", and "aaabbb" should be accepted, while "abb", "aab", or "bba" should be rejected. To achieve this, the development team decides to model the verification process using a Turing Machine (TM). The formal language describing the required pattern is:

L= $\{a^m b^m \mid m > 0\}$

Construct a Turing Machine for implementing the verification module.

51. A robotic printer in a research lab follows a strict pattern while printing characters on a tape. The printer's control logic can be modeled as a Turing Machine (TM). The rule followed by the printer is:
    It prints two extra 'a's before every sequence of 'b's, such that the number of 'b's is n, and the total number of 'a's is n + 2, where n > 0.
    Formally, the language printed by the machine is:
    L= $\{a^{n+2} b^n \mid n > 0\}$
52. A company named Binary Logic Systems Pvt. Ltd. is working on an automated verification module for validating binary message patterns. In this system, every valid message must follow a strict rule — it must begin with a sequence of 1's, followed by an equal number of 0's, and finally end with the same number of 1's. For example: Valid messages: 101, 110011, 111000111 and Invalid messages: 1001, 1101, 11100111.
    To test the accuracy of their message verification logic, the team decides to model the behavior using a Turing Machine (TM), since TMs can recognize such complex languages that require counting and matching patterns across multiple segments. The formal definition of the language is given as:
    L=$\{1^n 0^n 1^n \mid n>=1\}$
    Design a Turing Machine (TM) that accepts the given language.

Topic – Theoretical questions of Turing Machine, Decidability, and Complexity Theory

53. Discuss any five variants of the Turing Machine with a proper model diagram and transition function.
54. What are the required fields of an instantaneous description of a Turing machine?
55. What do you mean by recursively enumerable language?
56. Explain the difference between recursive and recursively enumerable languages.
57. If L1 and L2 are recursive language, then L1UL2 is a recursive language.
58. Explain Church thesis and why is a thesis not a theorem with explanation.
59. Explain Halting problem in Turing Machine.
60. If a language L and its complement L' both are recursively enumerable then prove that L is recursive**.**
61. Enlist the comparative study of Deterministic Finite Automata, Push Down Automata and Turing Machine based on their mathematical description, working, etc.
62. When do we say a problem is decidable? Give an example of undecidable problem?
63. Explain the difference between P and NP problems.